Aprendizaje no supervisado

Índice

Definición	01
Tipos de técnicas	02
Clustering	03
Reglas de asociación	04
Reducción de dimensiones	05
Pipeline	06

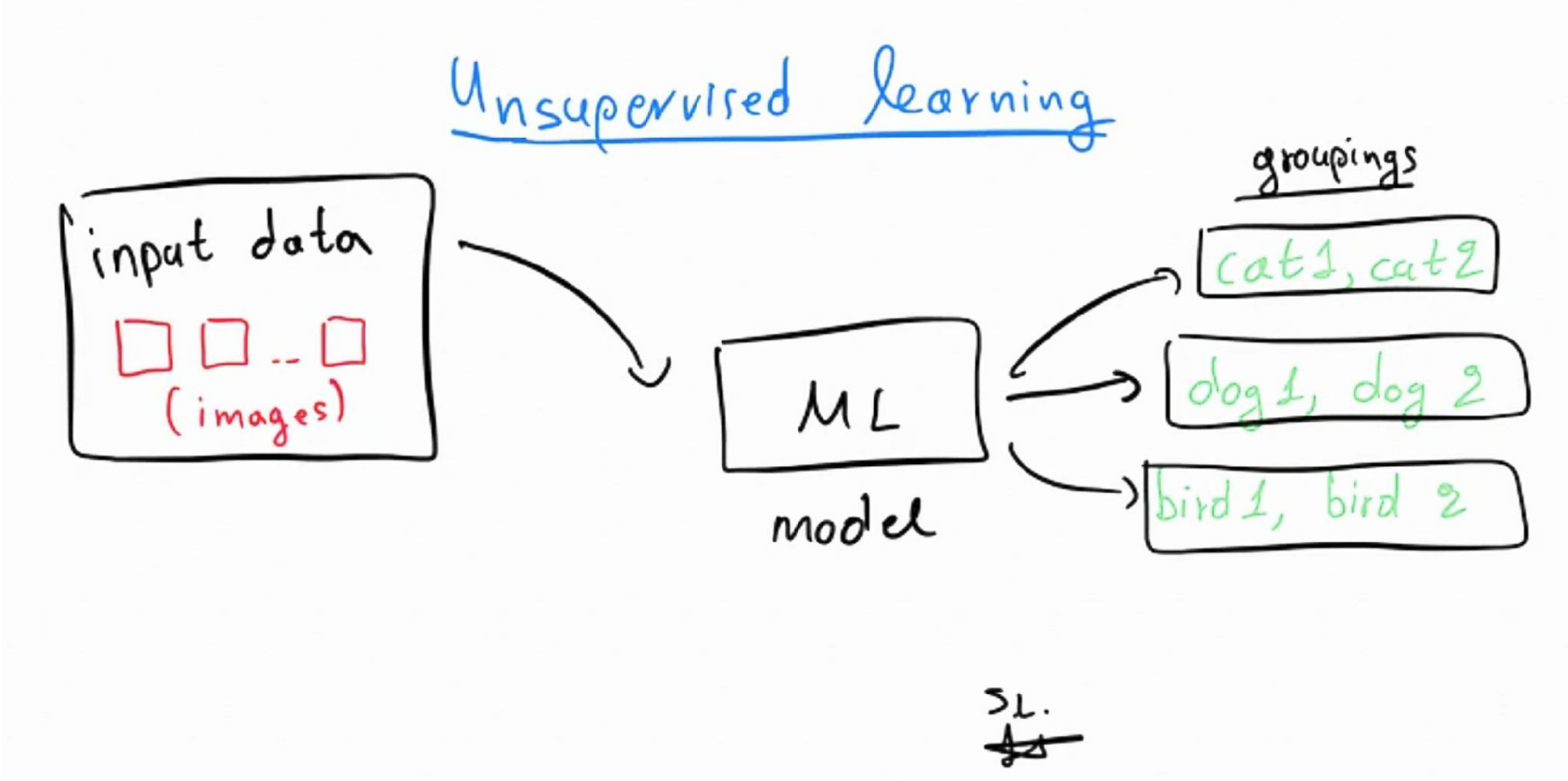
Aprendizaje no supervisado

El aprendizaje no supervisado es un grupo de técnicas de Machine Learning que trabajan sobre datos sin respuestas etiquetadas. A diferencia del aprendizaje supervisado, donde los modelos se entrenan utilizando resultados conocidos, el aprendizaje no supervisado se centra en encontrar patrones y relaciones en conjuntos de datos sin etiquetas preexistentes.

El principal objetivo del aprendizaje no supervisado es agrupar información no ordenada según similitudes y patrones sin ningún entrenamiento previo de los datos.

Este enfoque se utiliza a menudo para descubrir patrones ocultos o agrupamientos:
descripción de poblaciones, detección de anomalías, reconocimiento de objetos.

Aprendizaje no supervisado



Principales técnicas de aprendizaje no supervisado

1

Clustering

- Es una de las principales técnicas de aprendizaje no supervisado
- Consiste en agrupar conjuntos de datos de tal manera que los datos que sean el mismo grupo son más similares entre sí que a los de otros grupos
- Los algoritmos más comunes son: K-means, clustering jerárquico, DBSCAN, HBSCAN, mapas auto organizados (SOM)

2

Reglas de asociación

- Se emplea para encontrar asociaciones entre diferentes variables
- Se utiliza en análisis de cestas de mercado para identificar eventos que ocurre a la vez, por ejemplo: “Clientes que comprar A también compran B”
- Algoritmos: A priori, FP Growth

3

Reducción de dimensiones

- Reduce las variables de un set de variables en un número reducido de variables.
- Se agrupa de tal manera que las variables resultantes de la reducción recojan la maxima informacion posible recogida en el data set original.
- Las técnica de reducción de dimensión más comunes son Componentes Principales (PCA), t-SNE, LDA, UMAP

4

Detección de Anomalías

- Permite identificar patrones inusuales que no se ajustan al comportamiento esperado.
- Se utiliza para identificar transacciones fraudulentas, mantenimiento preventivo, monitorización de constantes vitales
- Técnicas más comunes: One Class SVM, Isolation Forest

Algunas aplicaciones
Aprendizaje no supervisado

Marketing

Identificar segmentos de clientes con comportamientos de compra similares sin conocimiento previo.
Esta segmentación permite estrategias de marketing más específicas y una interacción personalizada con el cliente.

Salud

Detección de anomalías para identificar registros de pacientes inusuales que podrían indicar actividad fraudulenta o condiciones críticas no diagnosticadas.

Banca

Detectar patrones en el comportamiento de gasto que pueden indicar transacciones fraudulentas.
Al analizar millones de transacciones, estos modelos pueden identificar valores atípicos que no se ajustan a la norma.
También se emplea para describir poblaciones de clientes.

Retail

Se utilizan reglas de asociación para el análisis de la cesta de la compra y comprender las afinidades de los productos. Este análisis ayuda a optimizar la colocación de productos en lineales y ayuda a definir estrategias de venta cruzada al identificar productos que se compran frecuentemente juntos.

Clustering: K-means

Su objetivo principal es agrupar un conjunto de datos en K clústeres, donde cada punto de datos pertenece al clúster con el centroide más cercano.

Cómo Funciona K-means:

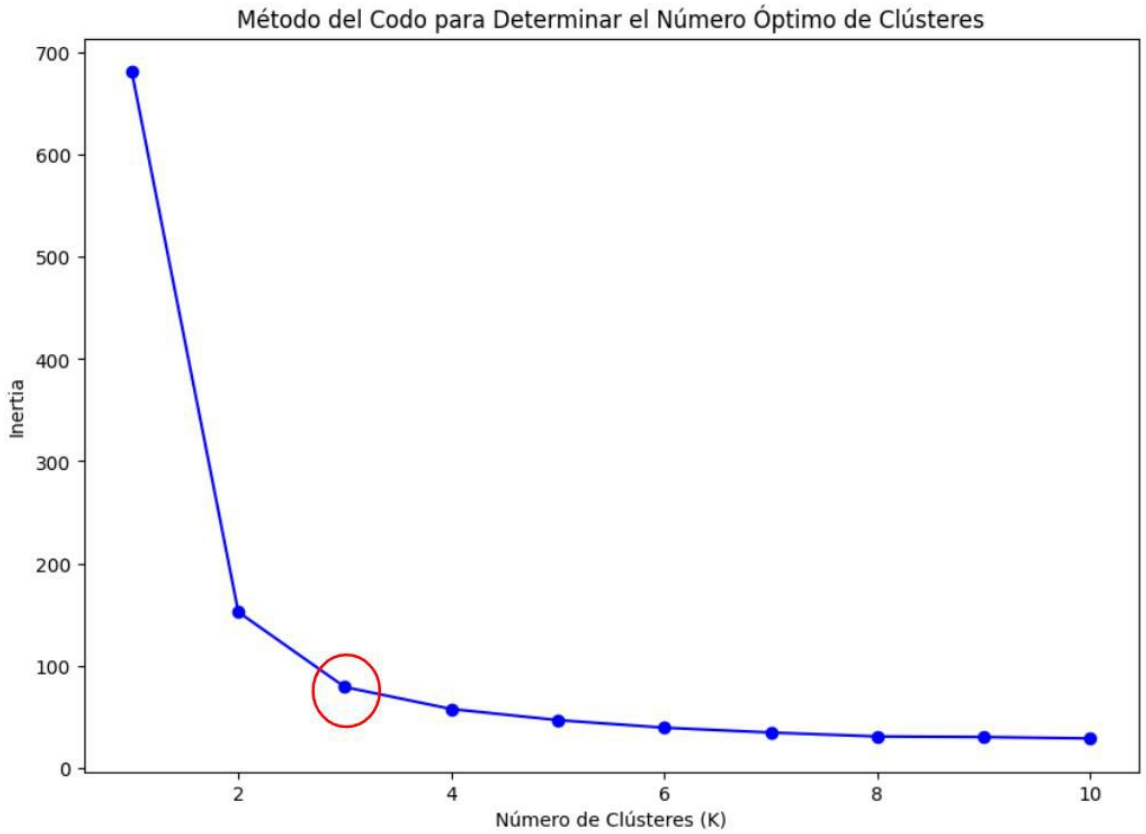
1. **Inicialización**
   Elige K centros de clústeres (centroides) de manera aleatoria.
2. **Asignación de Clústeres**
   Asigna cada punto de datos al centroide más cercano, formando K clústeres.
3. **Actualización de Centroides**
   Calcula los nuevos centroides como la media de todos los puntos de datos asignados a cada clúster
4. **Repetición**
   Repite los pasos 2 y 3 hasta que los centroides ya no cambien significativamente o se alcance un número máximo de iteraciones.

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X)
```

Clustering: K-means

Consideraciones: Hay que elegir a prioiri un número de clusters(K). Existe un método, método del codo (Elbow Method), que evalúa la suma de las distancias al cuadrado dentro del clúster (inertia) para diferentes valores de K. Esto nos ayuda a determinar un número de clusters óptimo.

Representando en una gráfica estas inertias para varios valores de k (se suele elegir de 2 a 10-12). >Cuando la curva se aplana es que hemos llegado al k óptimo.



Clustering: Jerárquico

El clustering jerárquico es una técnica de aprendizaje no supervisado utilizada para agrupar datos en clústeres con una estructura jerárquica. Este método organiza los datos en forma de un árbol o dendograma, donde cada nodo representa un clúster.

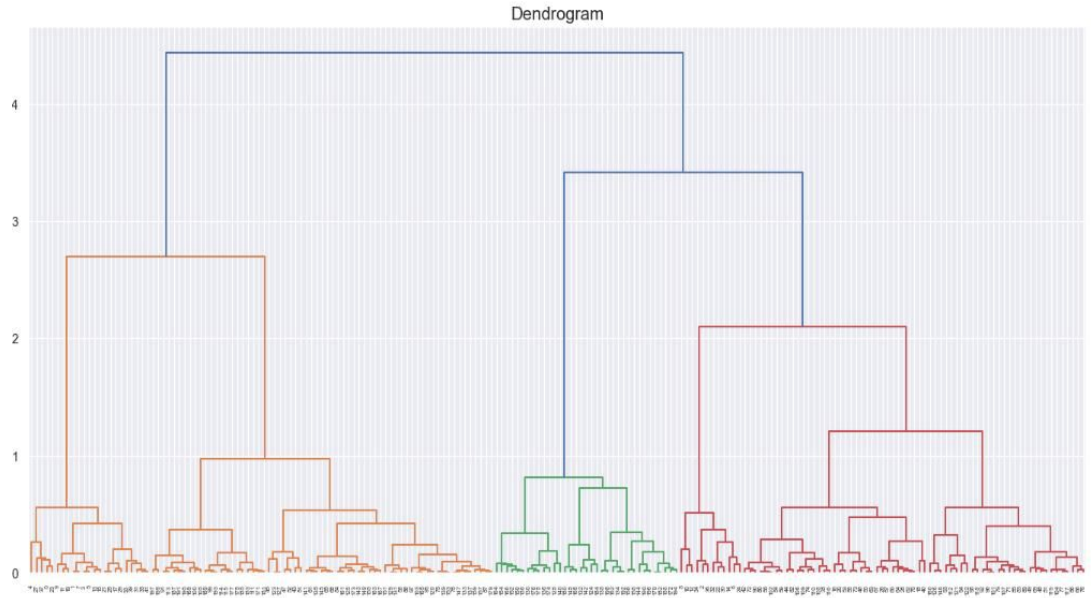
Pasos clave del clustering jerárquico:

1. **Inicialización**
   Cada punto de datos se considera un clúster individual al inicio.

2. **Cálculo de la distancia:**
   Se calcula la distancia entre cada par de clústeres. Existen varias formas de medir la distancia, como la distancia euclidiana, Manhattan, o cualquier otra métrica.

3. **Agrupación iterativa:**
   Los dos clústeres más cercanos se combinan para formar un nuevo clúster. Esto se repite hasta que todos los puntos de datos estén en un solo clúster o se alcance un criterio de parada predefinido.

4. **Construcción del dendograma:**
   El proceso de agrupación se representa en forma de un dendograma, que muestra cómo los clústeres se fusionan en cada paso.




```
agc = AgglomerativeClustering(n_clusters = 5, metric = 'euclidean', linkage = 'ward')
hc_labels = agc.fit_predict(X)
```

Clustering: DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN: consiste en agrupar puntos de datos en función de la densidad. No requiere predefinir el número de clústeres y es capaz de encontrar clústeres de forma arbitraria. Es especialmente útil para conjuntos de datos con formas irregulares y ruido.

Parámetros de DBSCAN:

1. **Epsilon (ε):** Define el radio de un vecindario alrededor de un punto de datos. Dos puntos son considerados vecinos si la distancia entre ellos es menor o igual a ε.
2. **MinPts:** El número mínimo de puntos necesarios para formar un clúster. Si un punto tiene al menos MinPts vecinos dentro de su radio ε, se considera un "punto central".

```
dbscan = DBSCAN(eps=0.1, min_samples=5)
dbscan.fit(X_norm)
dbs_labels = dbscan.labels_
```

Clustering: HDBSCAN (Hierarchical DBSCAN)

HDBSCAN extiende DBSCAN construyendo una jerarquía de clústeres a distintos niveles de densidad y seleccionando automáticamente los clústeres más estables. Elimina la necesidad de fijar epsilon (ε) manualmente y maneja mejor clústeres de densidad variable, algo que DBSCAN clásico no resuelve bien.

1 Construcción del árbol de densidad

Se calcula una jerarquía similar al clustering jerárquico, pero basada en densidad (mutual reachability distance) en lugar de distancia directa entre puntos.

2 min_cluster_size

Tamaño mínimo de puntos para que un grupo se considere un clúster válido — sustituye conceptualmente al par (ε, MinPts) de DBSCAN.

3 min_samples

Controla cuán conservador es el algoritmo al marcar puntos como ruido; valores más altos generan más puntos etiquetados como -1 (ruido).

4 Extracción de clústeres estables

HDBSCAN selecciona automáticamente el nivel de corte de la jerarquía que produce los clústeres más persistentes (Excess of Mass), sin intervención del usuario.

Importante: a diferencia de DBSCAN, HDBSCAN no requiere fijar ε, maneja densidad variable y asigna un score de estabilidad a cada clúster.

```
from hdbscan import HDBSCAN

hdbscan_model = HDBSCAN(min_cluster_size=15, min_samples=5)
hdb_labels = hdbscan_model.fit_predict(X)
```

Reglas de asociación

Las **reglas de asociación** se utiliza principalmente para descubrir relaciones interesantes entre variables en grandes conjuntos de datos.

Este método es particularmente efectivo en el análisis de la cesta de la compra, que ayuda a los retailers a comprender las relaciones entre las compras. Al identificar artículos que frecuentemente se compran de manera conjunta. Con ellos las empresas pueden optimizar la colocación de productos y las estrategias de venta cruzada.

Conceptos clave:

Soporte (support)

Indica la frecuencia con la que el conjunto de elementos aparece en el conjunto de datos.

- Soporte = Número de transacciones que contienen el conjunto de items(X) / Total de transacciones.

Confianza (confidence)

Mide la probabilidad de que se compre el artículo Y cuando se compra el artículo X.

Elevación (Lift)

Indica con qué frecuencia se compran juntos los artículos X e Y en comparación con lo que se esperaría si fueran independientes estadísticamente. Lift mayor que 1 indica una asociación positiva

En Python se puede utilizar una librería llamada mlextend que contiene varios modelos para aplicar reglas de asociación:

https://rasbt.github.io/mlxtend/user_guide/frequent_patterns/association_rules/

Aquí vamos a ver 3 ejemplos:

1. **A priori:**
   frequency = apriori(df,min_support = 0.15,use_colnames = True,verbose = 1)

2. **FP Growth:**
   frequency_fp=fpgrowth(df, min_support=0.3, use_colnames=True)

3. **Association_rules:**
   rules = association_rules(frequency,metric = 'lift',min_threshold = 1.1)

Reducción de dimensiones

- Es un proceso que permite reducir dimensiones de los datos. Es especialmente útil cuando se trabaja con **muchas dimensiones (variables)**
- **Curse of dimensionality** se conoce como la maldición de la dimensionalidad. Aunque contar con más dimensiones puede parecer ventajoso, en la práctica, pudiera dificultar el análisis y procesamiento de datos. Por ejemplo, dos puntos que en dos dimensiones están próximos entre sí al añadir más dimensiones se empieza a alejar.
- Estas técnicas permite recoger en **unas pocas variables gran parte de la información recogida en las n-dimensiones.**
- **PCA, t-SNE, LDA, UMAP y autoencoders** son las técnica más comúnmente empleadas.

Reducción de dimensiones: PCA

PCA: Funciona identificando las características más significativas que capturan la máxima varianza en los datos, reduciendo el número de dimensiones mientras conserva la esencia de los datos originales. Se puede interpretar como un cambio de ejes.

1. **Estandarización**
   El primer paso consiste en estandarizar el rango de las variables iniciales continuas para que cada una de ellas contribuya de manera equitativa al análisis.

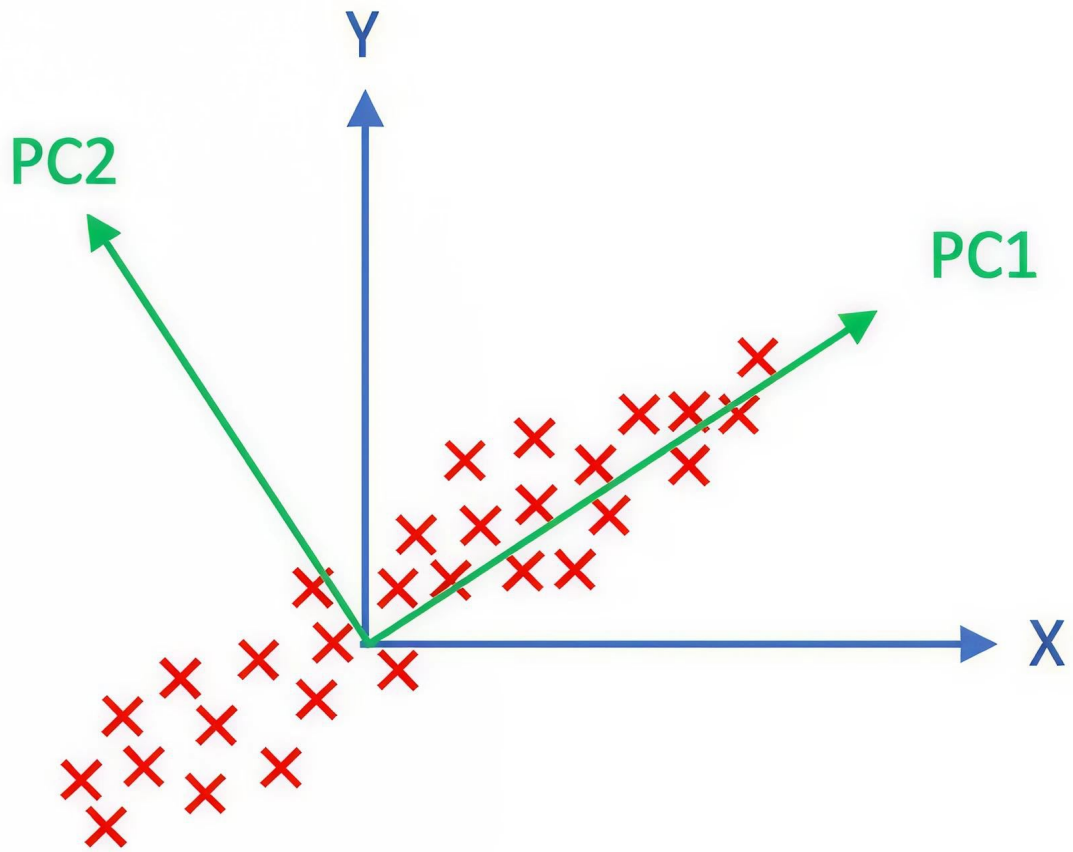
2. **Cálculo de la matriz de covarianza**
   El siguiente paso es calcular la matriz de covarianza para comprender cómo varían las variables respecto a la media entre sí.

3. **Valores y vectores propios**
   Se calculan los vectores propios y los valores propios de la matriz de covarianza para identificar los componentes principales.
   Los vectores propios indican la dirección de la mayor varianza en los datos, mientras que los valores propios definen su magnitud.

4. **Vector de características**
   Se selecciona un subconjunto de vectores propios que más contribuyen a la varianza para formar el vector de características.

Reducción de dimensiones: PCA

```
from sklearn.decomposition import PCA
# Configure PCA for 2 principal components
pca = PCA(n_components=2)
# Fit and transform the data
transformed_data = pca.fit_transform(original_data)
```



Reducción de dimensionalidad: t-SNE

El t-Distributed Stochastic Neighbor Embedding (t-SNE) se utiliza principalmente para la visualización de datos de alta dimensionalidad.

Al convertir las similitudes entre puntos de datos en probabilidades conjuntas y minimizar la divergencia de Kullback-Leibler entre diferentes puntos. Esta divergencia mide la distancia entre dos distribuciones y en este caso sirve para minimizar la pérdida de información. t-SNE permite una visualización efectiva en un espacio de baja dimensionalidad, como 2D o 3D.

Reducción de Complejidad

t-SNE reduce la complejidad de datos de alta dimensionalidad, facilitando su interpretación.

Identificación de Clústeres

Sobresale en la identificación de clústeres en los datos, lo que puede ser crucial para tareas como segmentación de clientes y detección de anomalías.

Sensibilidad a los Parámetros

El resultado de t-SNE es altamente sensible a la elección de parámetros, particularmente el parámetro de perplexity, que equilibra la atención entre los aspectos locales y globales de los datos.

Coste computacional

Como inconveniente, tSNE tiene un elevado coste computacional especialmente a medida que crece el tamaño del set de datos. Se recomienda en estas ocasiones trabajar con muestras.

Importante

t-SNE no es posible emplearlo como input de un modelo ya que no tiene función transform. Es decir que creado el modelo se pueda aplicar a un set de datos nuevo.
t-SNE es estocástico: sin fijar semilla, distintas ejecuciones pueden producir geometrías diferentes. Fijando random_state se logra reproducibilidad práctica, aunque no siempre exacta entre distintas versiones/implementaciones.

Principales parámetros:

n_components (default=2): Determina el número de dimensiones del espacio en el que se proyectarán los datos (por ejemplo, 2D o 3D).

perplexity (default=30.0): Controla el equilibrio entre los aspectos locales y globales de los datos. Generalmente, valores entre 5 y 50 funcionan bien, pero depende de los datos.

learning_rate (default=200.0): La velocidad de aprendizaje influye en la convergencia del algoritmo. Si es demasiado pequeña, la optimización será lenta, y si es demasiado alta, puede provocar resultados inestables.

n_iter (default=1000): Número máximo de iteraciones para la optimización. Si no se observan mejoras, el algoritmo puede detenerse antes.

```
from sklearn.manifold import TSNE
import pandas as pd

# Standardize the data
standardized_data = (data - data.mean()) / data.std()

# Initialize and run t-SNE
tsne = TSNE(n_components=2, perplexity=30, n_iter=1000)
tsne_results = tsne.fit_transform(standardized_data)
```

Reducción de dimensionalidad: Otras técnicas

Autoencoders

Son un tipo de red neuronal que funcionan aprendiendo de una representación comprimida y codificada de los datos, para luego reconstruir la salida a partir de esta codificación y hacerla coincidir con la entrada original lo más fielmente posible. Los autoencoders son útiles en donde es necesario identificar patrones o características en los datos sin etiquetas explícitas.

Se utilizan en: procesamiento de imágenes, la detección de anomalías y la extracción de características

LDA

Linear Discriminant Analysis (LDA) busca proyectar los datos originales (que pueden tener muchas dimensiones) en un espacio de menor dimensionalidad, maximizando al mismo tiempo la separabilidad entre las clases.LDA, además de reducir la dimensionalidad como PCA (Principal Component Analysis), tiene en cuenta las etiquetas de las clases, se puede decir, por tanto, que es un algoritmo supervisados.

Se utiliza por ejemplo en reconocimiento facial, clasificación de textos y análisis biomédico.

Detección de anomalías

1. La detección de anomalías ayuda a identificar patrones inusuales que no se ajustan al comportamiento esperado.

2. La detección de anomalías funciona analizando puntos de datos y diferenciando valores atípicos de los eventos normales.

3. Dentro de los algoritmos de aprendizaje no supervisado, existen técnicas que permiten la detección de anomalías, sin necesidad de datos etiquetados.
Esto resulta especialmente útil cuando etiquetar los datos como normales o anómalos es poco práctico, costoso o imposible debido al volumen o la complejidad de la información.

Caso de uso:
Detección del fraude, prevención de fallos en equipos, en salud para monitorizar constantes vitales de los pacientes y predecir posibles problemas de salud, detección del fraude en siniestros de seguros, amenzas de cyberseguridad detectando trafico inusual.

Detección de anomalías: Isolation Forest

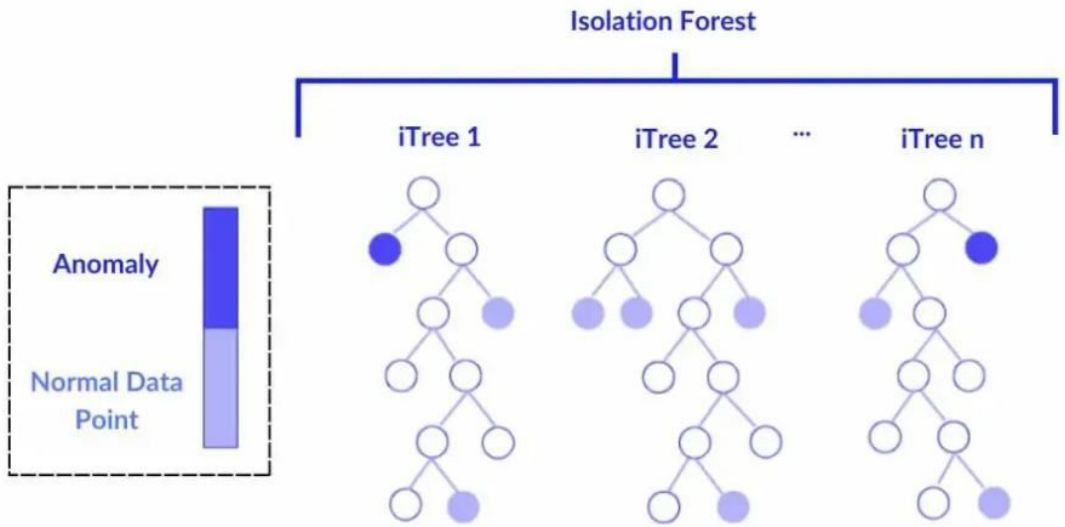
Isolation Forest:
Esta técnica aísla las anomalías empleando árboles aleatorios,en cada árbol se divide el espacio de manera aleatoria por medio de una variable y un valor de corte.
Las observaciones anómalas necesitan menos divisiones para quedar solos.

- Funciona eficazmente cuando se tienen muchos datos
- No requiere escalado de los datos
- Rápido y robusto

```
from sklearn.ensemble import IsolationForest
model = IsolationForest(n_estimators=100,
contamination=0.05, random_state=42)
model.fit(X)
```

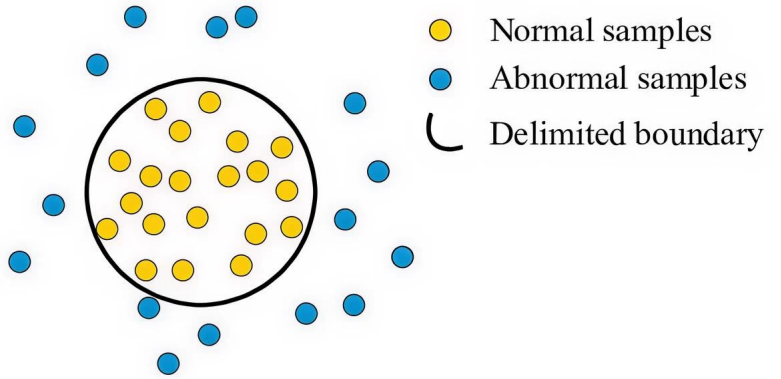
N_estimators: número de árboles

contamination: es el % de observaciones que queremos etiquetar como anomalía



Detección de anomalías: One Class SVM

One Class SVM construye un frontera (hiperplano) que encierra la mayoría de los datos y todo lo que quede fuera de esa frontera se considera una anomalía.



- Se utiliza cuando se requiere una precisión alta
- Cuando no se tienen muchas observaciones debido a su alto coste computacional
- Requiere escalado de los datos

```
from sklearn.svm import OneClassSVM
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Entrenar el modelo One-Class SVM
ocsvm = OneClassSVM(kernel='rbf', gamma='auto', nu=0.05)
ocsvm.fit(X_scaled)
```

kernel: Tipo de función núcleo que define la forma de la frontera. Por defecto es 'rbf' (radial), ideal para fronteras no lineales. Otros valores: 'linear', 'poly', 'sigmoid'.

gamma: Controla la influencia de cada punto de entrenamiento. 'scale' (por defecto) usa 1 / (n_features * var(X)). 'auto' usa 1 / n_features. También puedes pasar un valor numérico.

nu: Proporción de errores permitidos y mínimo de vectores de soporte. Por ejemplo, nu=0.05 permite hasta un 5% de anomalías.

¿Qué es un pipeline?

¿Qué es un pipeline en Python?

Un pipeline en Python se refiere a una secuencia de operaciones o transformaciones que se aplican a los datos de manera encadenada. Es como una tubería donde los datos fluyen de una etapa a otra, siendo procesados y transformados en cada paso.

¿Por qué usar pipelines?

Legibilidad: El código se vuelve más conciso y fácil de entender.

Reutilización: Puedes crear pipelines personalizados y reutilizarlos en diferentes partes de tu código.

Eficiencia: Al encadenar operaciones, puedes evitar crear variables intermedias innecesarias.

Mantenimiento: Es más fácil modificar o agregar nuevas etapas a un pipeline.

```
pipeline = Pipeline([
    ('MyHotEncoder', FunctionTransformer(MyHotEncoder)),      # Paso 1: gestiona variables categoricas
    ('quitar_nulos', FunctionTransformer(quitar_nulos)),      # Paso 2: quita registros con nulos
    ('normalizar', FunctionTransformer(normalizar))           # Paso 3: Normalizar
])
X_transf = pipeline.fit_transform(X)
```

GridSearchCV

GridSearchCV es una herramienta de Scikit-Learn que automatiza el proceso de optimización de hiperparámetros para modelos de ML.

Su objetivo es encontrar la mejor combinación de parámetros mediante una búsqueda exhaustiva en un espacio definido por el usuario.
Lo hace entrenando el modelo con diferentes combinaciones de parámetros especificados en una cuadrícula **(param_grid)** y evaluando el rendimiento usando una métrica (por ejemplo, precisión, coeficiente de silhouette, etc.) definida mediante el argumento **scoring.** Esto hace que dado un espacio de parámetros podamos encontrar el modelo que mejor funciona.

El proceso se lleva a cabo con **validación cruzada (cv),** lo que garantiza que los resultados no se basen en un solo conjunto de entrenamiento y evaluación, sino que aprovechan divisiones del conjunto de datos para lograr una evaluación más robusta. Al final, GridSearchCV devuelve el mejor modelo según los criterios especificados, junto con los valores óptimos de hiperparámetros.

Otras alternativas a GridSearchCV:

- **RandomizedSearchCV**
  Pertenece al paquete Scikit-Learn y dados unos rangos va eligiendo parámetros de manera aleatoria
- **Optuna**
  Hace una optimizacion bayesiana

GridSearchCV

```
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import make_scorer, silhouette_score

kmeans = KMeans(random_state=77)

param_grid = {
    'n_clusters': [ 3, 4, 5, 6, 7, 8, 9],  # Probar diferentes valores de clústeres
    'init': ['k-means++', 'random'],  # Métodos de inicialización
    'n_init': [10, 20],  # Número de inicializaciones
    'max_iter': [300, 500]  # Iteraciones máximas
}

# cv indica el número de grupos que hace para el cross validation

def my_silhouette_scorer(estimator, X, y=None):
    labels = estimator.predict(X)
    return silhouette_score(X, labels) if len(set(labels)) > 1 else float('nan')

grid_search = GridSearchCV(
    estimator=kmeans,
    param_grid=param_grid,
    scoring = my_silhouette_scorer,
    cv=None,
    n_jobs=-1,
    verbose=2
)
```

Futuro del aprendizaje no supervisado

Key milestones

Desarrollo de técnicas de reducción de dimensionalidad: Técnicas como PCA y t-SNE han sido fundamentales. Ayudan a simplificar conjuntos de datos complejos, facilitando su análisis.

Avances en el análisis Cluster: Algoritmos como K-means, DBSCAN y el clustering jerárquico han evolucionado para manejar conjuntos de datos diversos de manera más eficiente, proporcionando segmentaciones más precisas.

Progresos en redes neuronales: El auge de los autoencoders y las Redes Generativas Antagónicas (GANs) ha ampliado los límites de lo que el aprendizaje no supervisado puede lograr, desde la generación de datos hasta el aprendizaje de características.

Impacto Computación Cuántica

Mejora de los algoritmos de Clustering: La computación cuántica facilita el desarrollo de algoritmos de clustering más sofisticados, que pueden identificar patrones y similitudes en los datos de manera más rápida y precisa.

Optimización de la selección de características: Aprovechando la mecánica cuántica, estos sistemas pueden procesar vastos conjuntos de datos de manera eficiente, seleccionando características relevantes sin intervención humana.

Rapidez en la detección de anomalías: Los algoritmos de aprendizaje no supervisado mejorados por la computación cuántica destacan en la identificación de valores atípicos o anomalías en los datos.

Casos de uso

Marketing y segmentación de clientes: Las empresas utilizan estos algoritmos para analizar datos de clientes e identificar segmentos distintos. Esto permite desarrollar estrategias de marketing más específicas y experiencias personalizadas para los clientes.

Diagnósticos en el sector sanitario: En el ámbito de la salud, estos algoritmos examinan grandes volúmenes de datos médicos para identificar patrones inusuales que pueden indicar enfermedades, a menudo antes de que los síntomas sean clínicamente evidentes.

Detección de anomalías financieras: Las instituciones financieras emplean estos algoritmos para identificar actividades sospechosas, como fraude o lavado de dinero.

Lecturas adicionales:

https://levelup.gitconnected.com/unsupervised-ml-5-association-rule-mining-explained-from-market-basket-to-recommendation-systems-17af455d49f2

https://www.kaggle.com/code/xvivancos/market-basket-analysis

https://medium.com/@prathik.codes/anomaly-detection-with-isolation-forest-an-introduction-1be7c5a69414